

# Lecture 12: Sequence-to-Sequence & Attention

**Instructor:** Stephanie Schoch

CSCI 680: Natural Language Processing  
Department of Computer Science, College of William & Mary  
October 2, 2026



# Lecture Outline

- Recap
- Generation
  - Autoregressive generation
  - Seq2Seq generation
- Encoder-Decoder
- Attention

# Recap

# Long Short-Term Memory (LSTM)

We have a sequence of inputs  $x^{(t)}$ , and we will compute a sequence of hidden states  $h^{(t)}$  and cell states  $c^{(t)}$ . On timestep  $t$ :

**Forget gate:** controls what is kept vs forgotten, from previous cell state

**Input gate:** controls what parts of the new cell content are written to cell

**Output gate:** controls what parts of cell are output to hidden state

**New cell content:** this is the new content to be written to the cell

**Cell state:** erase (“forget”) some content from last cell state, and write (“input”) some new cell content

**Hidden state:** read (“output”) some content from the cell

**Sigmoid function:** all gate values are between 0 and 1

$$f^{(t)} = \sigma \left( W_f h^{(t-1)} + U_f x^{(t)} + b_f \right)$$

$$i^{(t)} = \sigma \left( W_i h^{(t-1)} + U_i x^{(t)} + b_i \right)$$

$$o^{(t)} = \sigma \left( W_o h^{(t-1)} + U_o x^{(t)} + b_o \right)$$

$$\tilde{c}^{(t)} = \tanh \left( W_c h^{(t-1)} + U_c x^{(t)} + b_c \right)$$

$$c^{(t)} = f^{(t)} \circ c^{(t-1)} + i^{(t)} \circ \tilde{c}^{(t)}$$

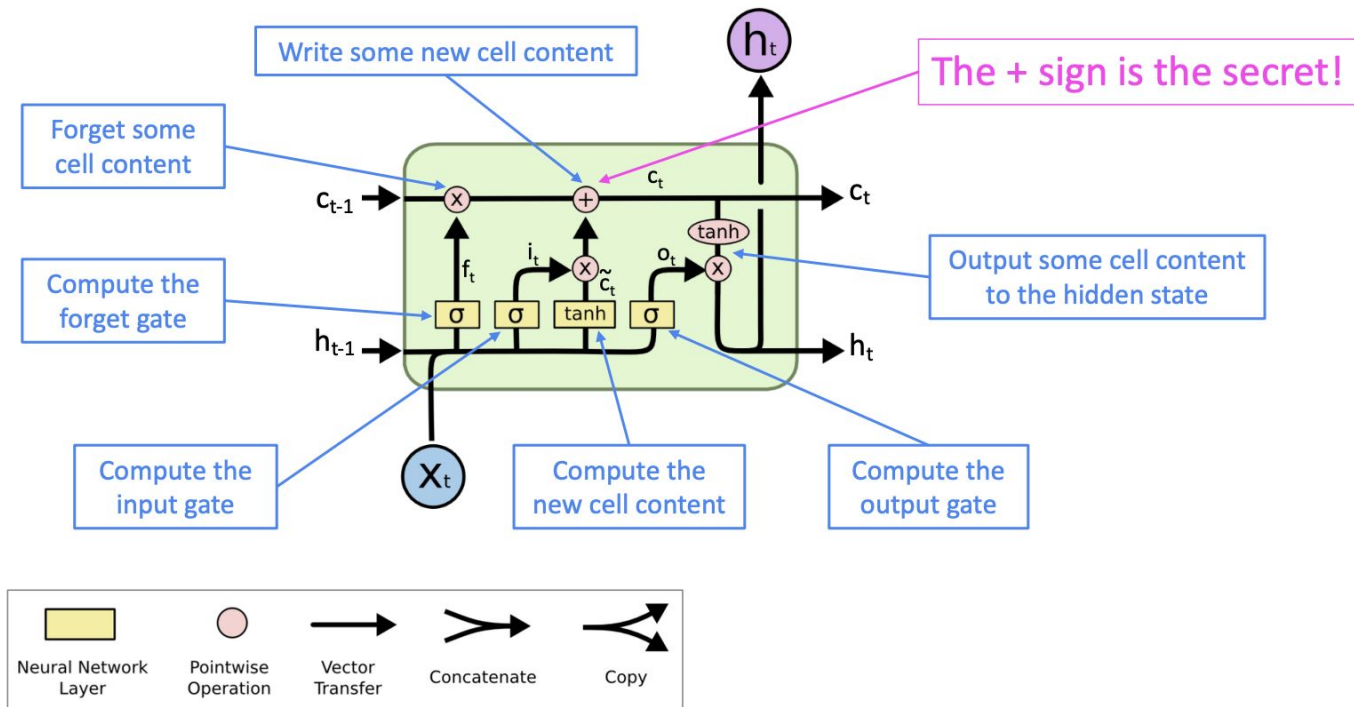
$$h^{(t)} = o^{(t)} \circ \tanh c^{(t)}$$

All these are vectors of same length  $n$

Gates are applied using element-wise (or Hadamard) product:  $\odot$

# Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



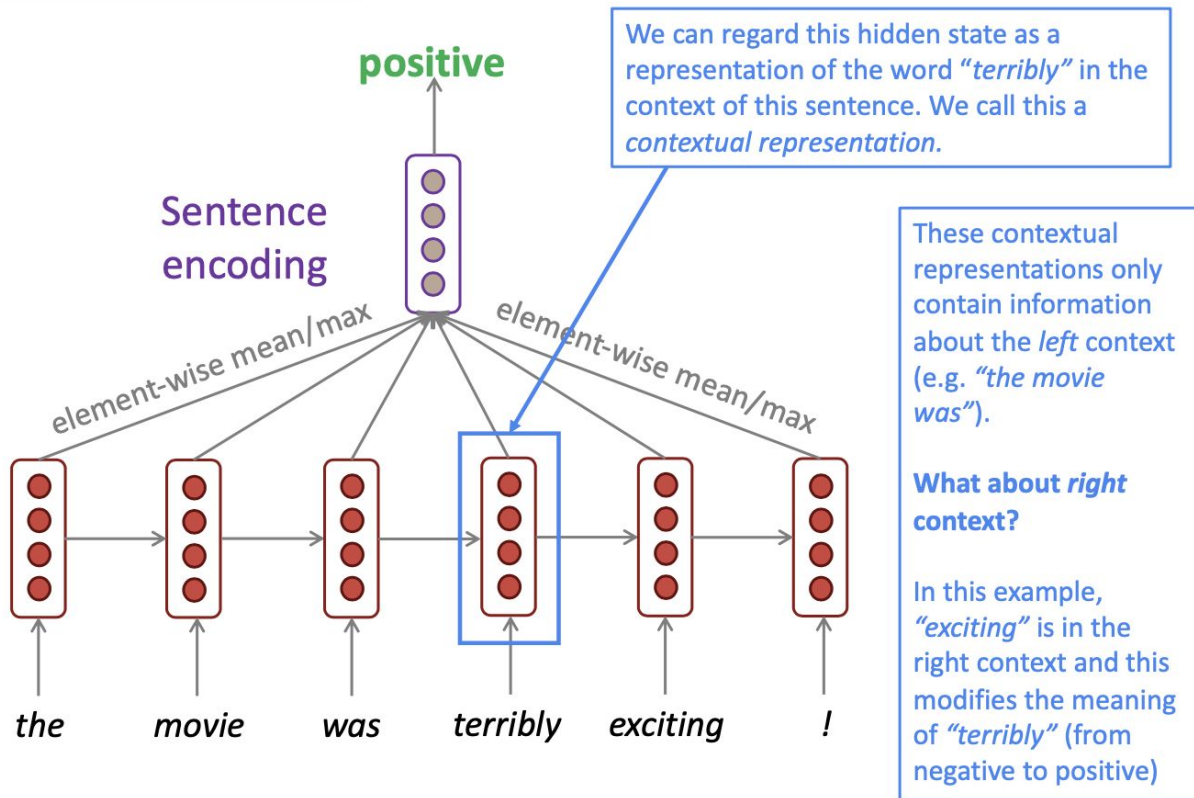
Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

# How does LSTM solve vanishing gradients?

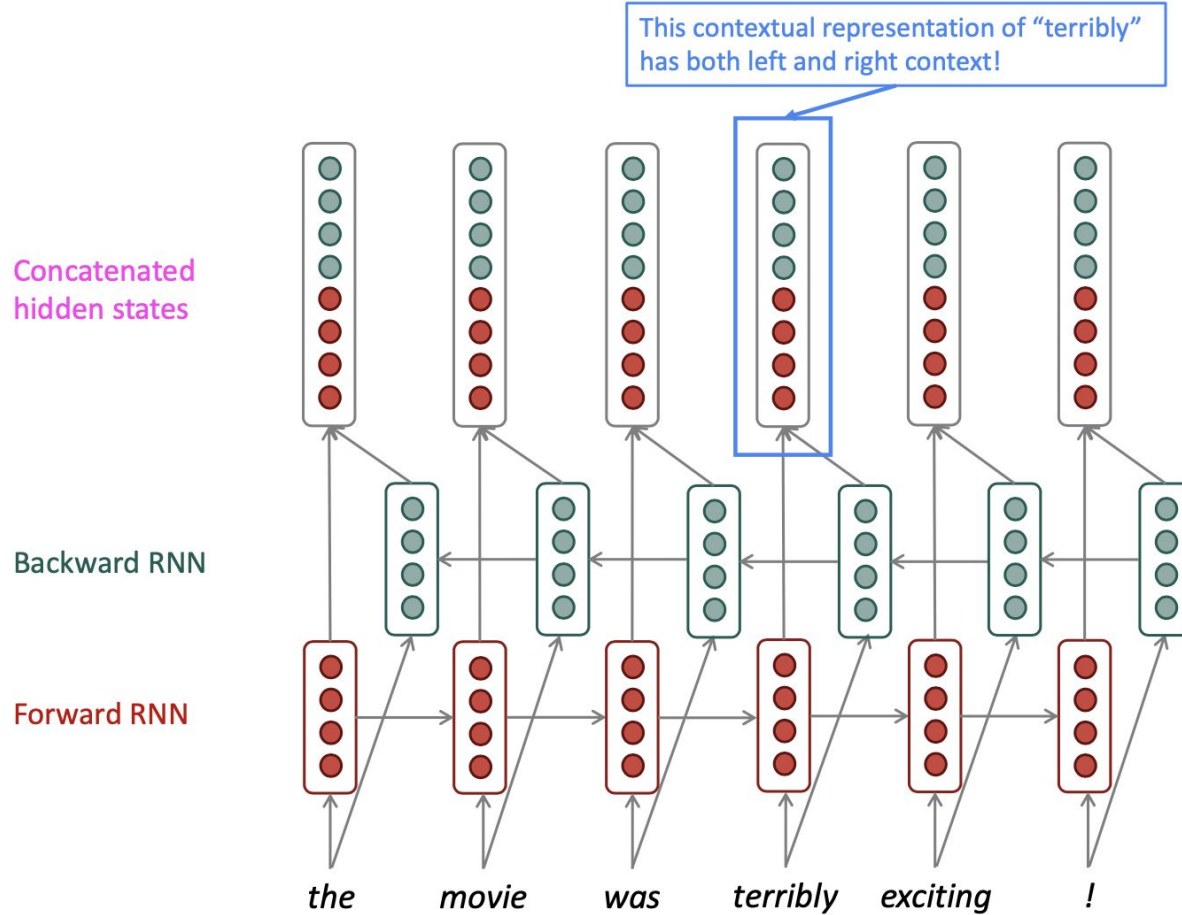
- The LSTM architecture makes it easier for the RNN to **preserve information over many timesteps**
  - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely.
  - In contrast, it's harder for a vanilla RNN to learn a recurrent weight matrix  $\mathbf{W}_h$  that preserves info in the hidden state
  - In practice, you get about 100 timesteps rather than about 7
- LSTM doesn't guarantee that there is no vanishing (or exploding) gradient, but it does provide an easier way for the model to learn long-distance dependencies

# Bidirectional RNNs

Task: Sentiment Classification



# Bidirectional RNNs



# Bidirectional RNNs

On timestep  $t$ :

This is a general notation to mean “compute one forward step of the RNN” – it could be a vanilla, LSTM or GRU computation.

Forward RNN  $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

Backward RNN  $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

} Generally, these two RNNs have separate weights

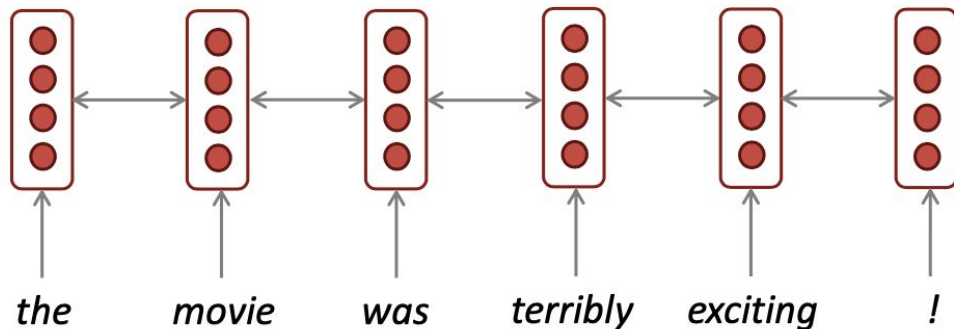
Concatenated hidden states

$$\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$$

We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

# Bidirectional RNNs

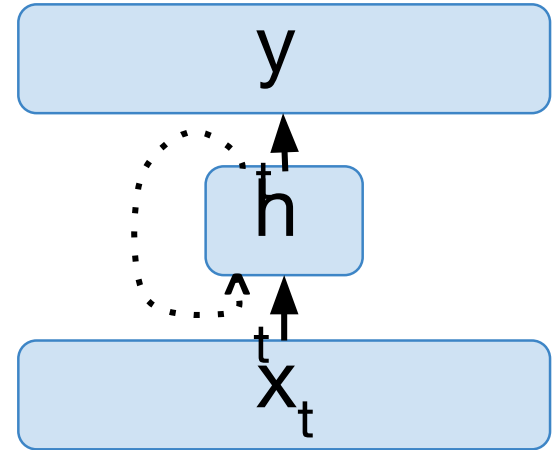
- Note: bidirectional RNNs are only applicable if you have access to the entire input sequence
- They are not applicable to Language Modeling, because in LM you only have left context available.
- If you do have entire input sequence (e.g., any kind of encoding), bidirectionality is powerful (you should use it by default).



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states

# Summary: RNNs

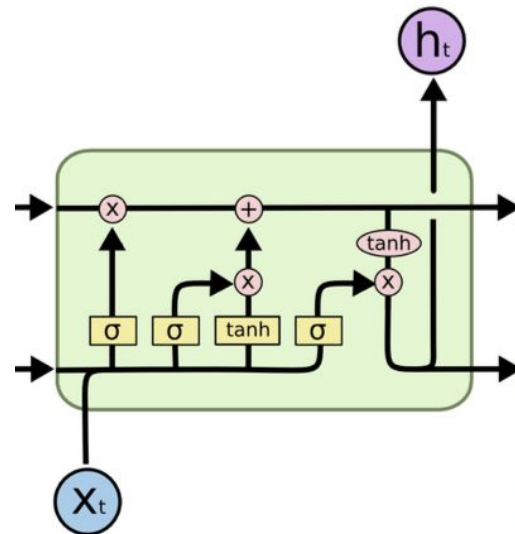
- Recurrent Neural Networks processes sequences one element at a time
- RNNs do not have:
  - the limited context problem of n-gram models
  - the fixed context limitation of feedforward LMs
  - since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence
- But training RNNs is hard:
  - Vanishing gradient problem
  - LSTMs address it by incorporating a memory



# Summary: LSTMs

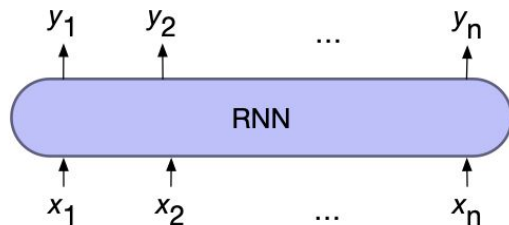
- The LSTM architecture makes it much easier for an RNN to preserve information over many timesteps
  - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely
- In 2013–2015, LSTMs started achieving state-of-the-art results

Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
- LSTMs became the dominant approach for most NLP tasks

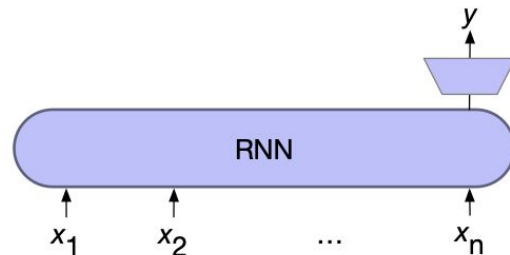


# Generation

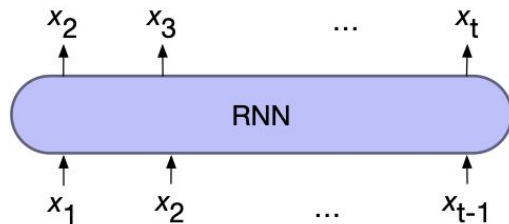
# Four architectures for NLP tasks with RNNs



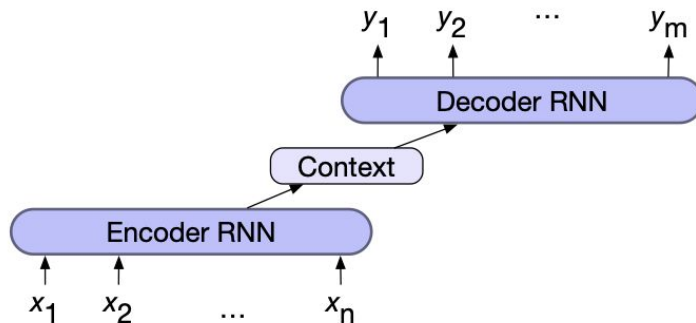
a) sequence labeling



b) sequence classification

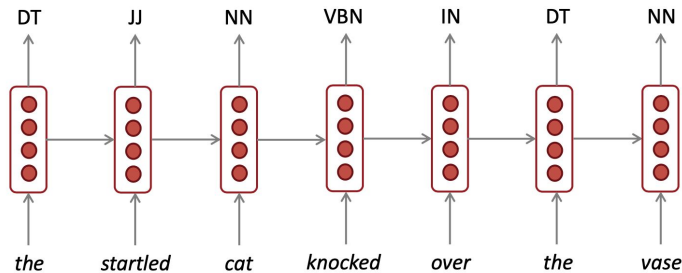


c) language modeling

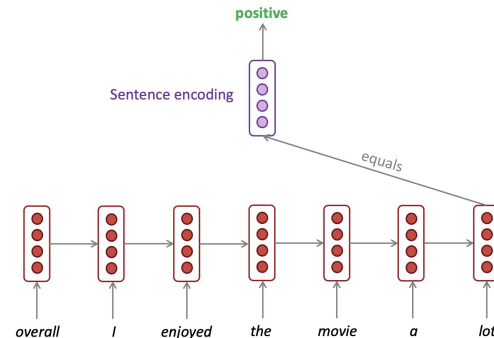


d) encoder-decoder

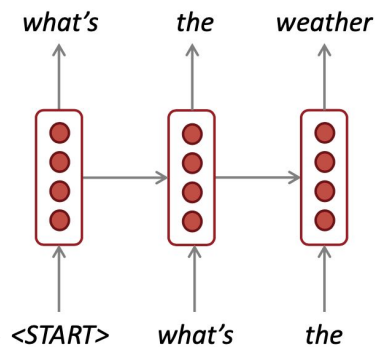
# Four architectures for NLP tasks with RNNs



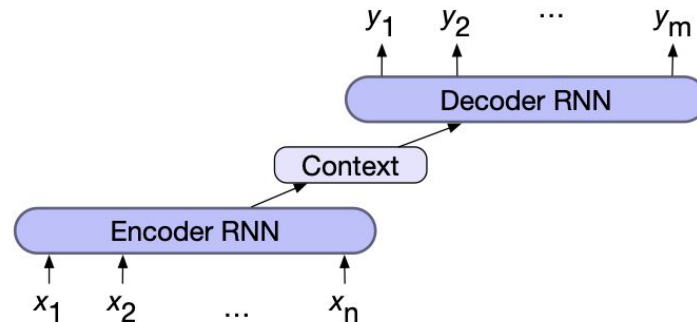
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder

# Generation with RNN-LMs

Start with beginning of sentence token `<s>`

1. Sample the next word from the softmax distribution
2. Use the word embedding for the sampled word as the input to the network at the next time step

Repeat until end of sentence token `</s>` or a maximum length is reached.

**Autoregressive generation:** Generate one token at a time, conditioning each next-token prediction on the tokens generated so far.

# RNN-LMs are Autoregressive Models

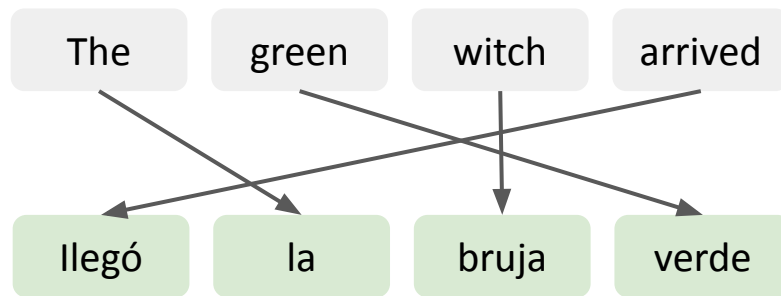
- Model that predicts a value at time based on a function of the previous values at times  $t-1$ ,  $t-2$ , and so on
- Word generated at each time step is conditioned on the word selected by the network from the previous step
- State-of-the-art generation approaches are all autoregressive!
  - Machine translation, question answering, summarization

**Key technique:** prime the generation with the most suitable context

**Idea:** Provide rich task-appropriate context than simply  $\langle s \rangle$

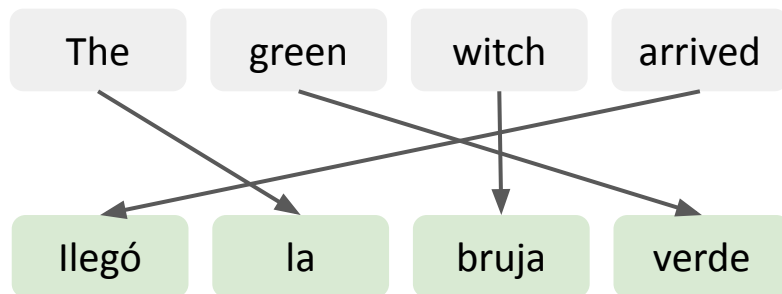
# Neural Machine Translation

- Sequence Generation Problem  
(as opposed to sequence classification)
  - $\mathbf{x}$  = Source sequence of length  $n$
  - $\mathbf{y}$  = Target sequence of length  $m$
- Different from regular generation from an LM
  - Since we expect the target sequence to serve a specific utility (translate the source)



**Sequence-to-Sequence (Seq2seq)**

# Sequence-to-Sequence Generation



- Mapping between a token in the input and a token in the output can be very indirect.
- In some language verb appears:
  - at the beginning of the sentence; e.g. Arabic, Hawaiian
  - at the end; e.g. Hindi
  - between the subject and the object; e.g. English

Need to summarize the entire context!

# Sequence-to-Sequence is Versatile!

Sequence-to-sequence is useful for *more than just MT*

- Many NLP tasks can be phrased as sequence-to-sequence:
  - **Summarization** (long text  $\rightarrow$  short text)
  - **Dialogue** (previous utterances  $\rightarrow$  next utterance)
  - **Parsing** (input text  $\rightarrow$  output parse as sequence)
  - **Code generation** (natural language  $\rightarrow$  Python code)
  - **Style transfer** ( $\text{text}_{\text{style1}} \rightarrow \text{text}_{\text{style2}}$ )

Sequence-to-Sequence Modeling: **Encoder-Decoder Network**

# Example Task: Stylized Text Generation

| Dataset     | Stylistic Task     | Domain                  | Annotation |
|-------------|--------------------|-------------------------|------------|
| Flickr      | Romantic→Humorous  | Image Captions          | Manual     |
| Shakespeare | Shakespeare→Modern | Literature, SparkNotes  | Automatic  |
| GYAFC-FR    | Informal→Formal    | Yahoo Answers (Online)  | Manual     |
| GYAFC-EM    | Informal→Formal    | Yahoo Answers (Online)  | Manual     |
| Biased-word | Subjective→Neutral | Wikipedia (Online)      | Automatic  |
| Fluency     | Disfluent→Fluent   | Telephone Conversations | Manual     |



Informal to Formal:

**Informal:** *Gotta see both sides of the story.*

**Formal:** *You have to consider both sides of the story.*

# Sequence-to-Sequence Models

**Seq2Seq Models:** given an input sequence, capable of generating output that is contextually appropriate & of arbitrary length

## Key idea:

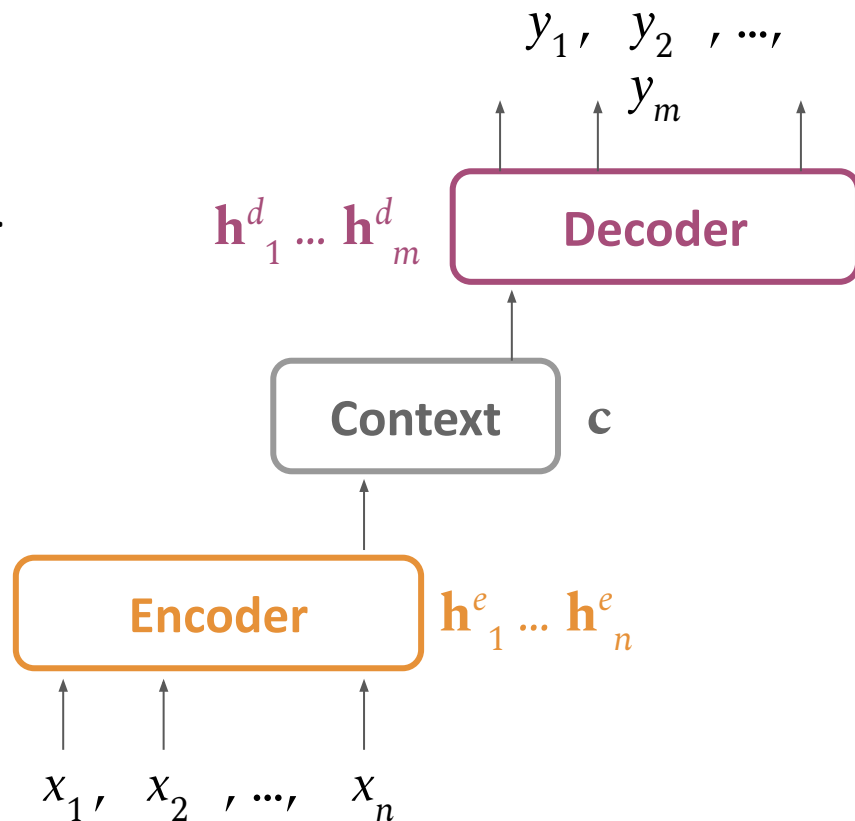
- Use of an **encoder network** that takes an input sequence and creates a contextualized representation of it, often called the context.
- This representation is then passed to a **decoder network** which generates a task-specific output sequence.

**Encoder** summarizes the input; **decoder** generates the output.

# Encoder-Decoder Network for Sequence-to-Sequence Modeling

# Encoder-Decoder Network: Components

1. An **encoder** that accepts an input sequence,  $x_{1:n}$ , and generates a corresponding sequence of contextualized representations,  $\mathbf{h}_{1:n}^e$ .
2. A **context vector**,  $\mathbf{c}$ , which is a function of  $\mathbf{h}_{1:n}^e$ , and conveys the essence of the input to the decoder.
3. A **decoder**, which accepts  $\mathbf{c}$  as input and generates an arbitrary length sequence of hidden states  $\mathbf{h}_{1:m}^d$ , from which a corresponding sequence of outputs  $y_{1:m}$ , can be obtained



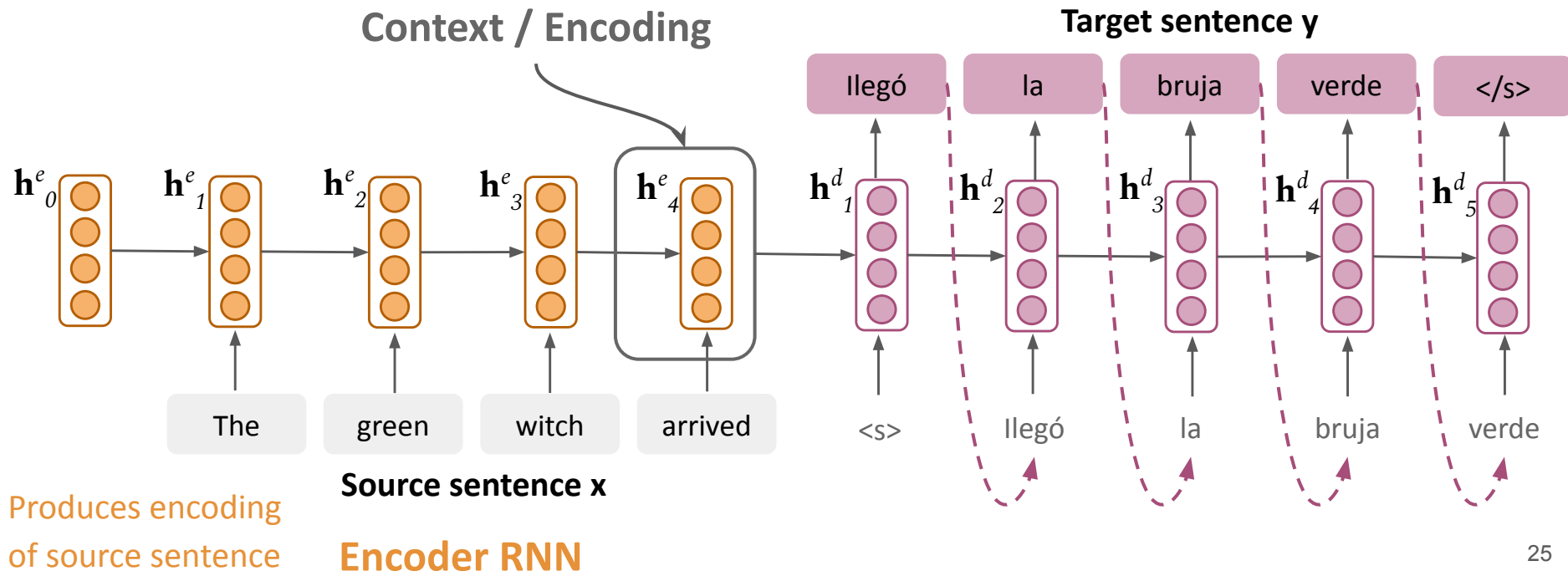
# Encoder-Decoder Network

Represents input sequence.

Provides initial hidden state for Decoder RNN

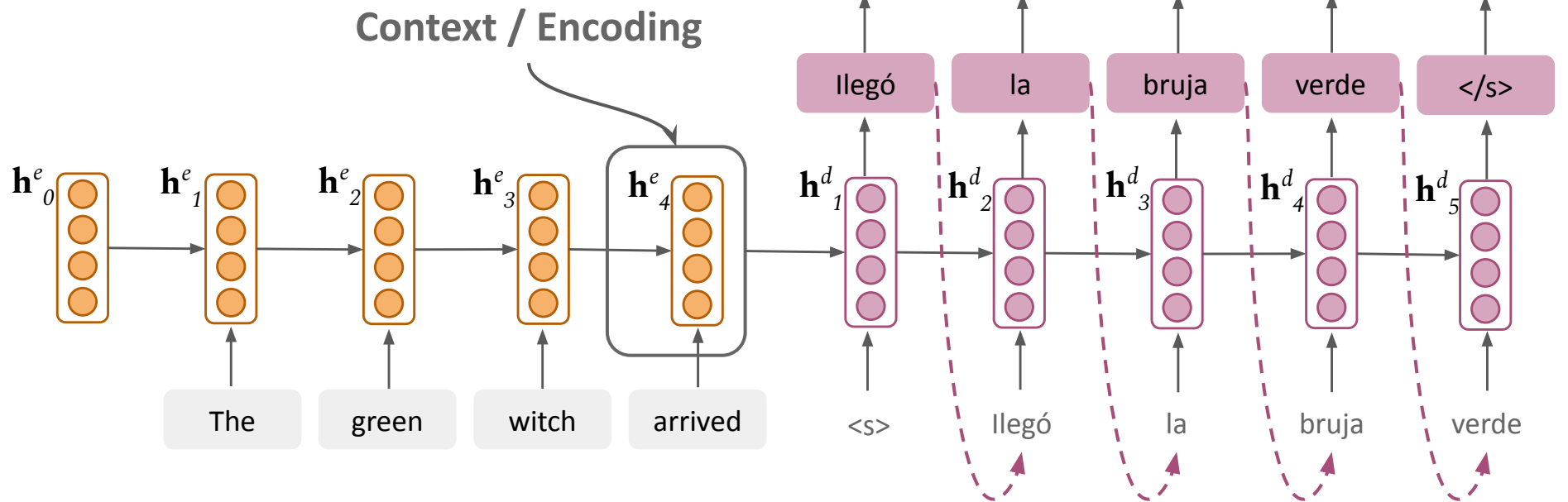
Language Model that produces the target sentence conditioned on the encoding

## Decoder RNN



# Encoder-Decoder: Loss

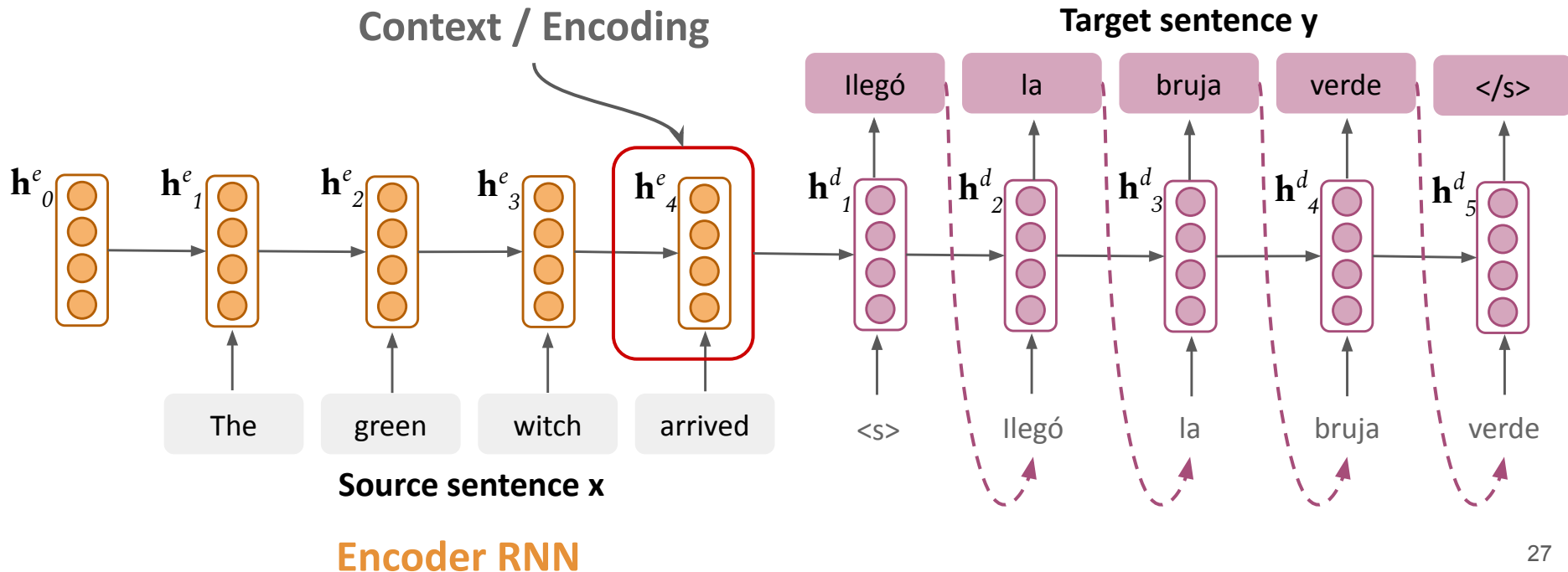
**Loss**  $\rightarrow L(\theta) = \frac{1}{T} \sum_{t=1}^T L_t(\theta) = L_1(\theta) + L_2(\theta) + L_3(\theta) + L_4(\theta) + L_5(\theta)$



# Encoder-Decoder Network: Information Bottleneck

Needs to capture all of the information from the source sentence: **bottleneck!**

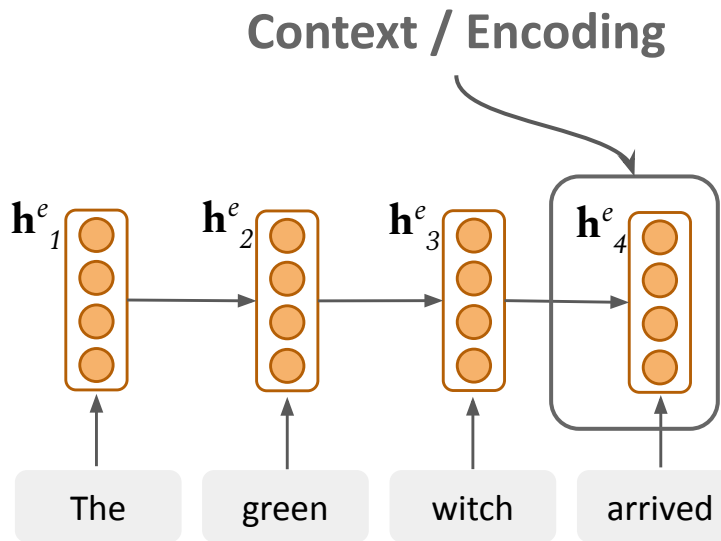
Decoder RNN



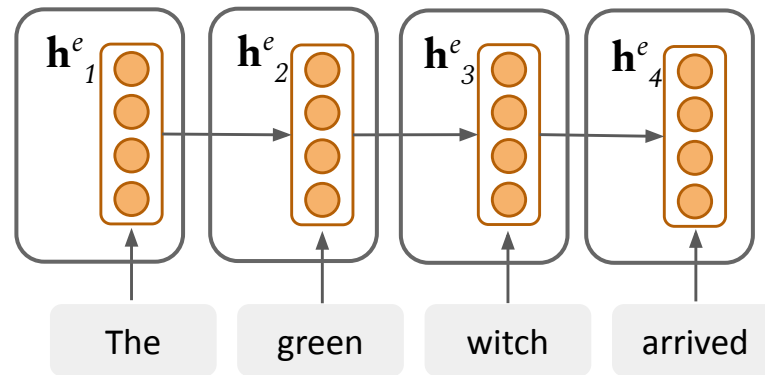
# Attention

# Information Bottleneck: A Solution

Encoder RNN



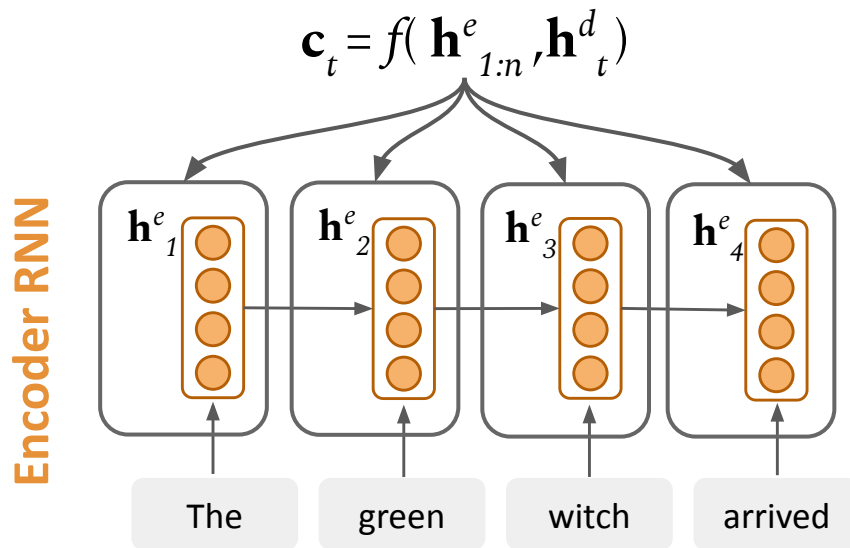
What if we had access to all hidden states?



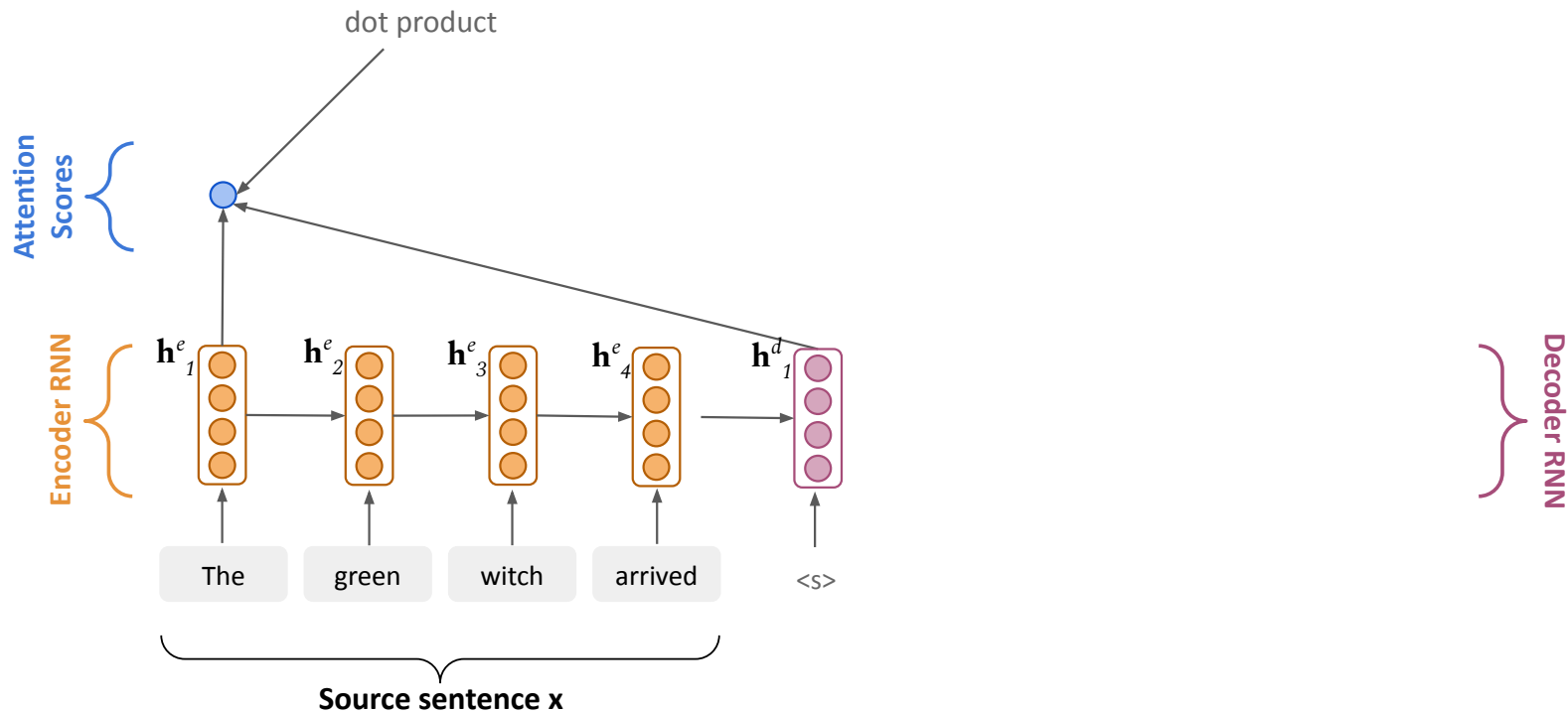
**Core Idea of Attention:** on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence

# Attention Mechanism

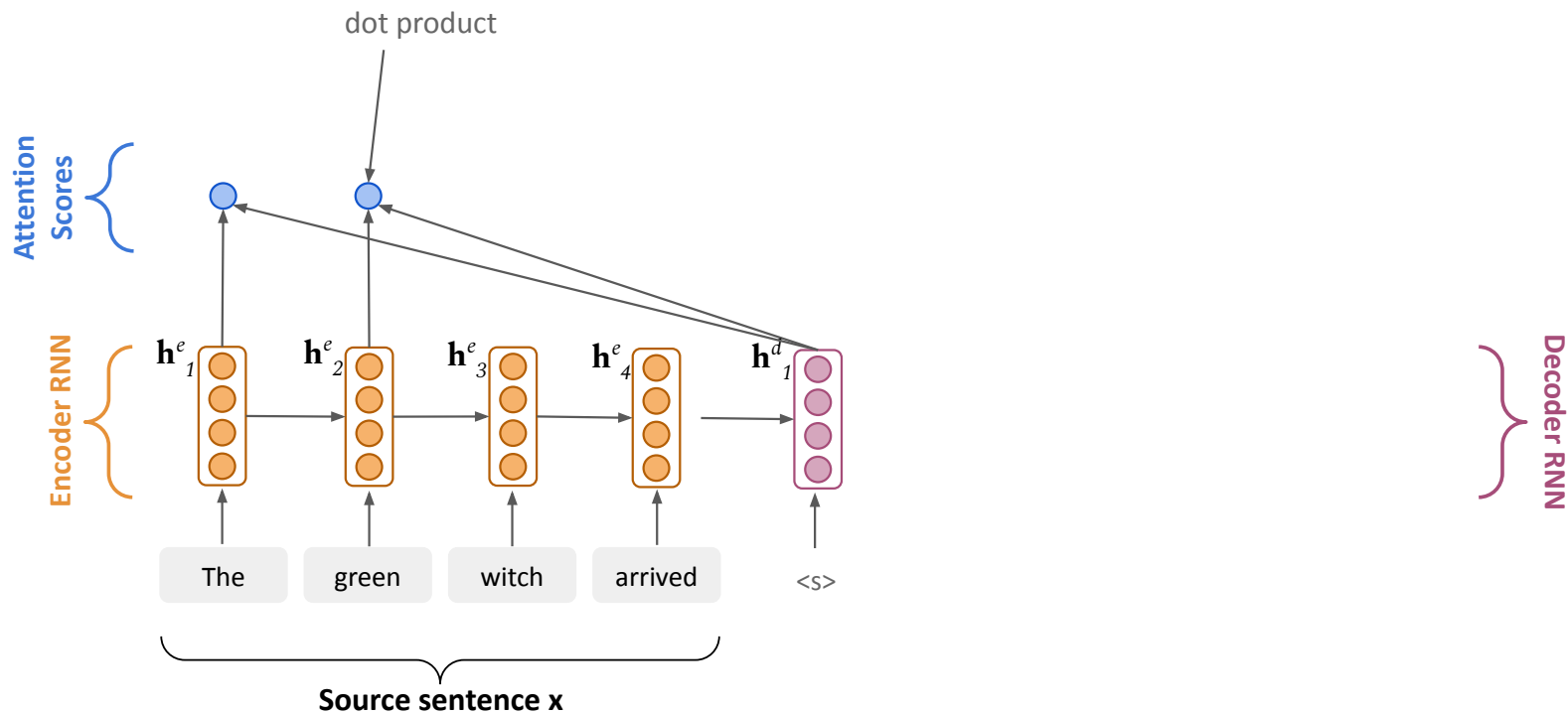
- Attention allows the decoder to focus on a particular part of the source sequence at each time step
- Attention context vector  $\mathbf{c}_t$ 
  - Weighted sum of the encoder hidden states
  - A different context vector at each decoder timestep
  - Weights emphasize source information relevant to the next output token (i.e. **attend** to the relevant information)



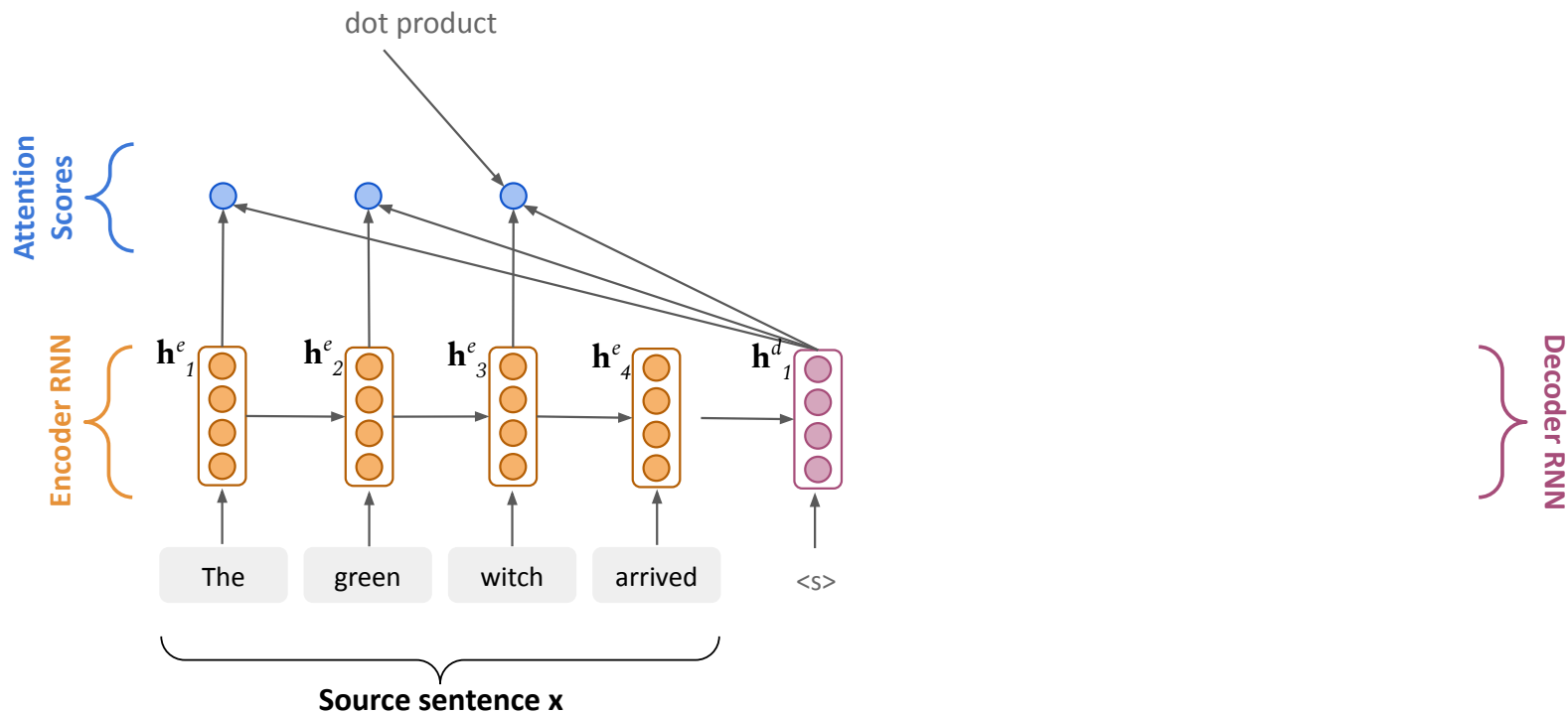
# Seq2seq with Attention



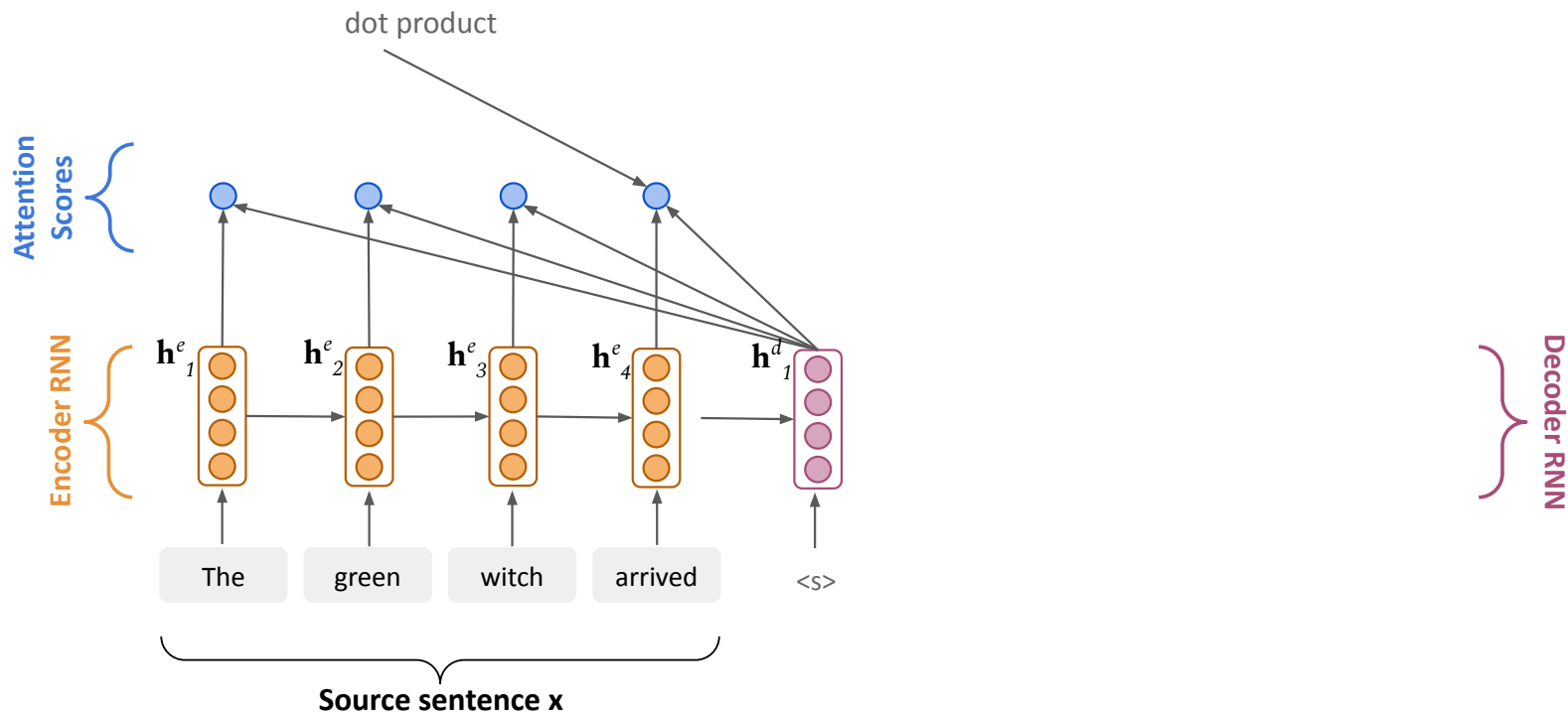
# Seq2seq with Attention



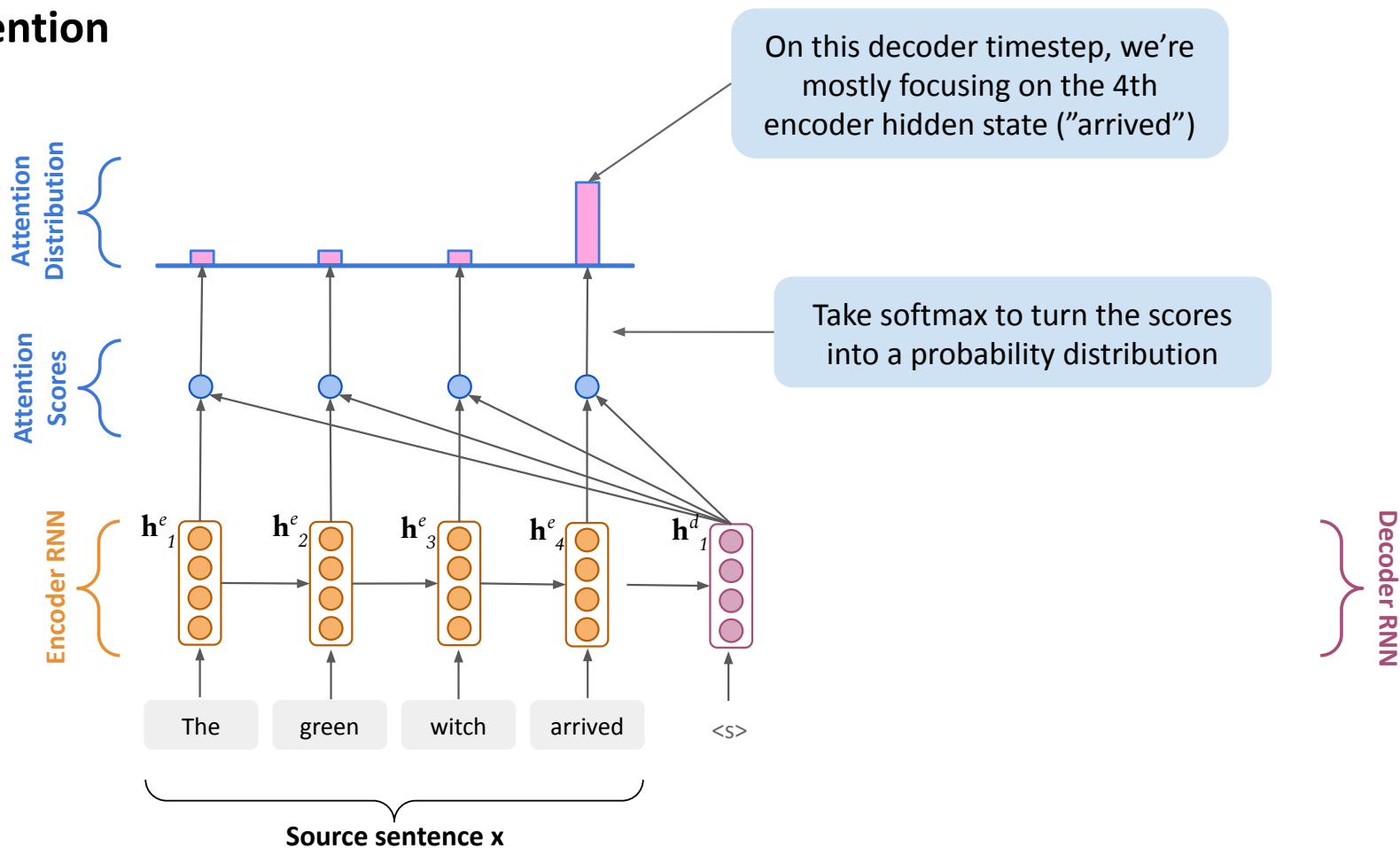
# Seq2seq with Attention



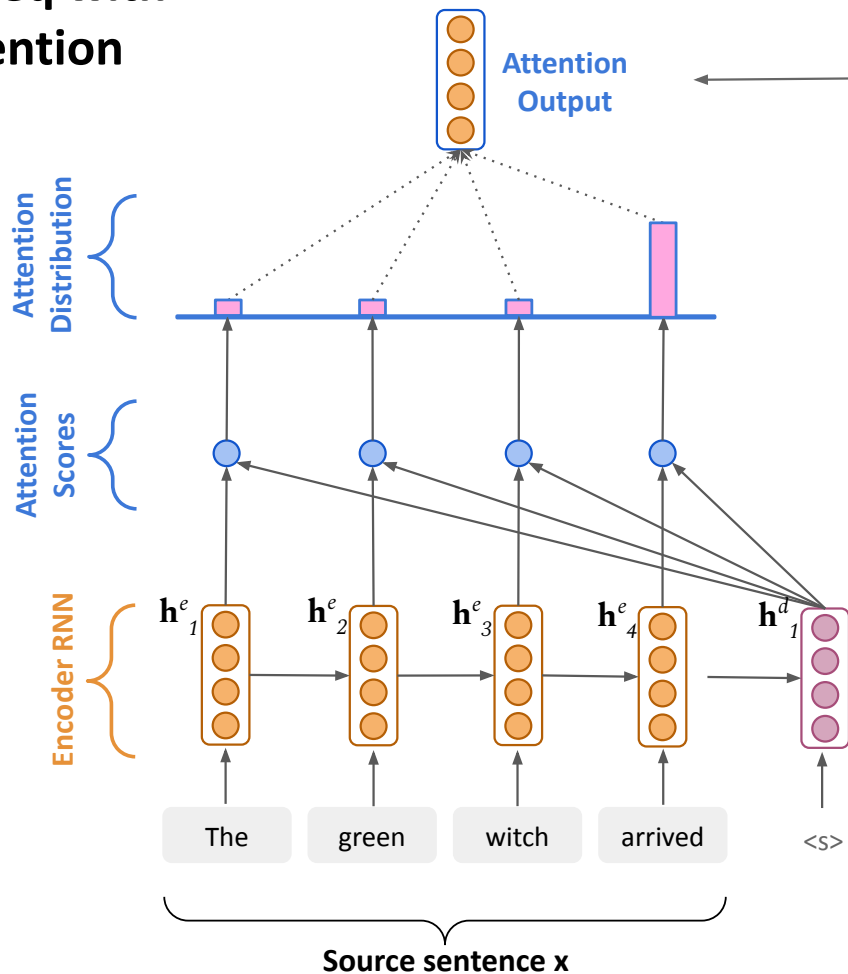
# Seq2seq with Attention



# Seq2seq with Attention



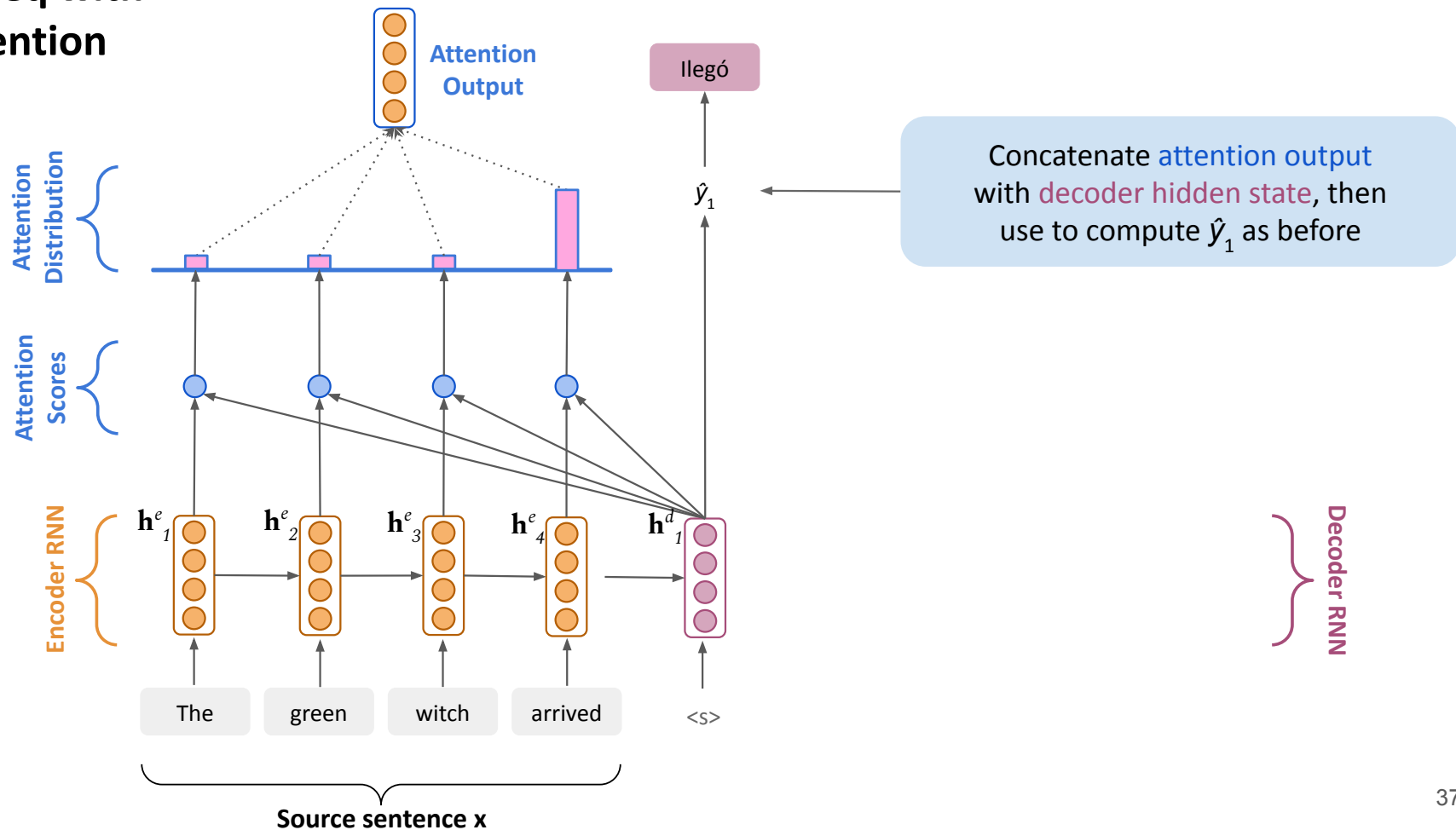
# Seq2seq with Attention



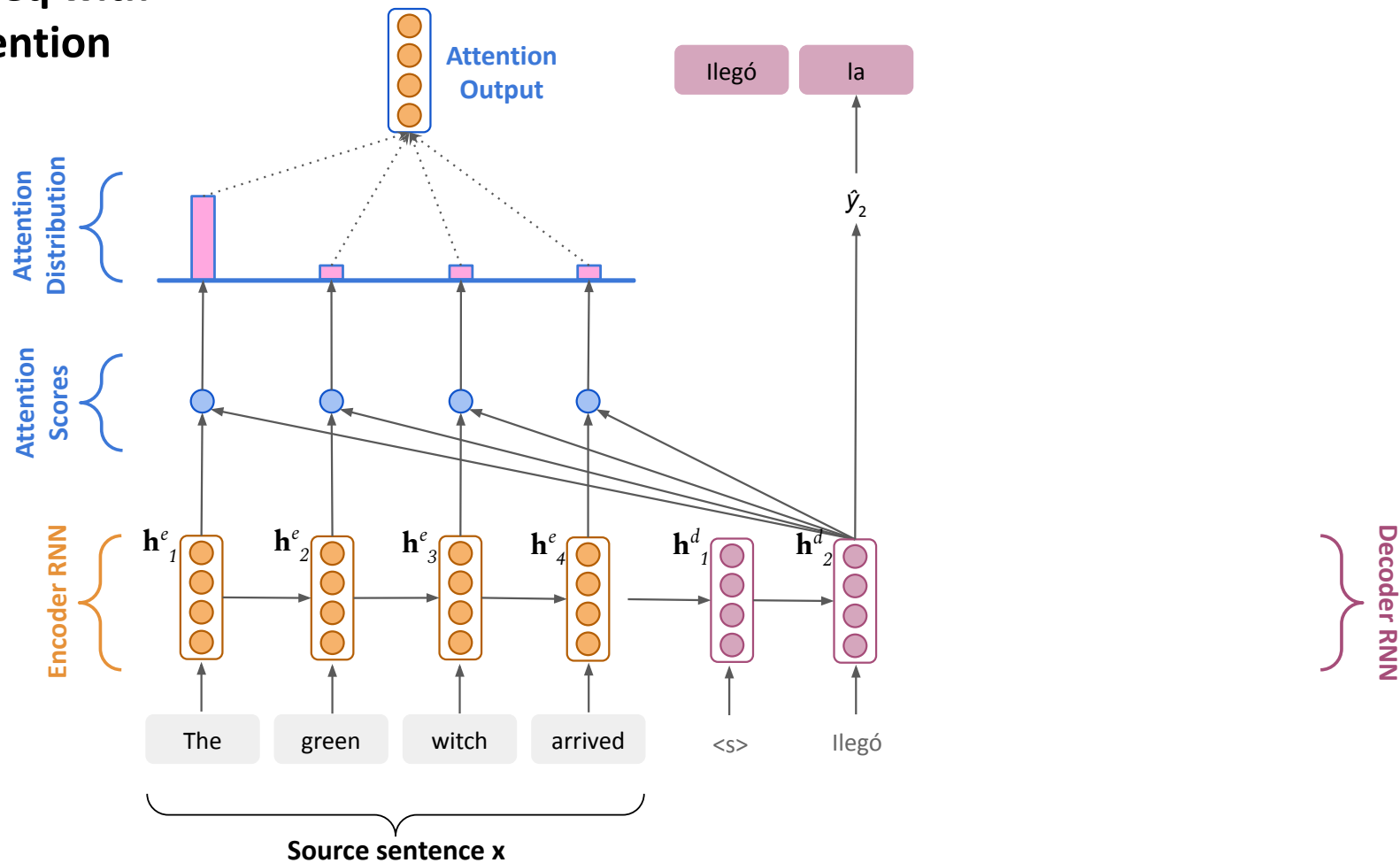
Use the **attention distribution** to take a **weighted sum** of the **encoder hidden states**. The **attention output** mostly contains information from the **hidden states** that received **high attention**.

Decoder RNN

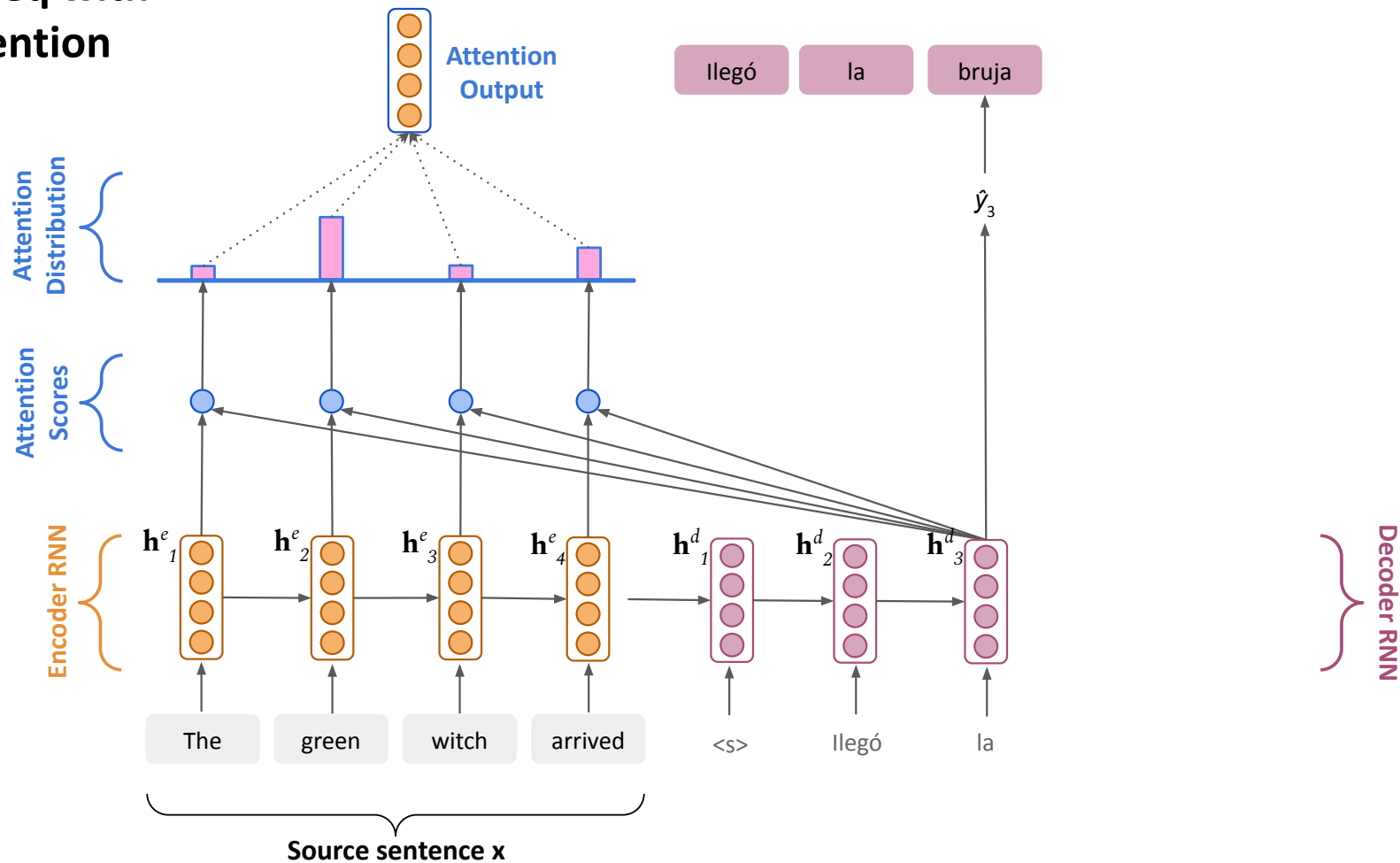
# Seq2seq with Attention



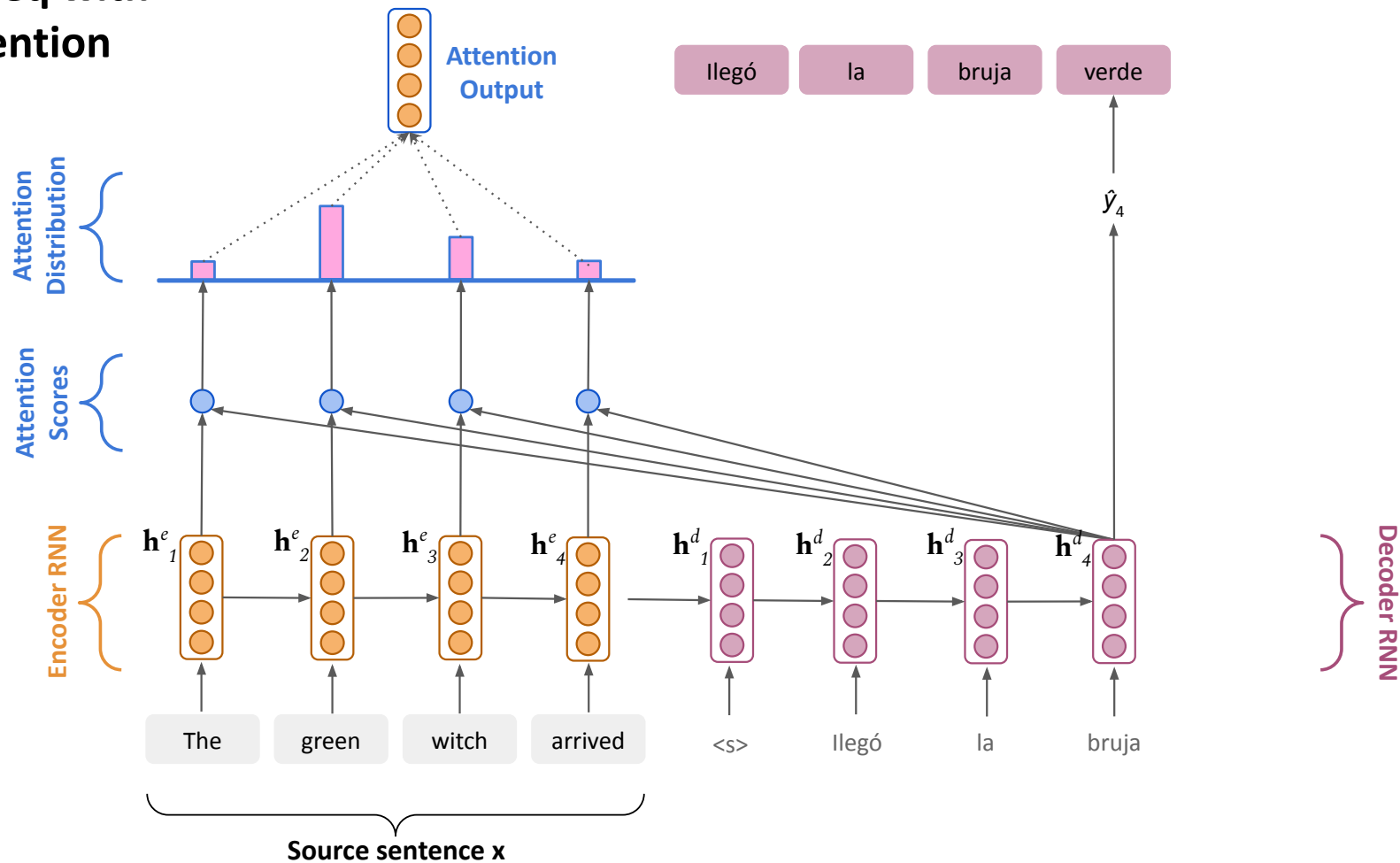
# Seq2seq with Attention



# Seq2seq with Attention



# Seq2seq with Attention



# Attention: Equations

- We have encoder hidden states  $\mathbf{h}_1^e, \dots, \mathbf{h}_n^e \in \mathbb{R}^h$
- On timestep  $t$ , we have decoder hidden state  $\mathbf{h}_t^d \in \mathbb{R}^h$
- We get the attention scores  $\mathbf{s}_t$  for this step:

$$\mathbf{s}_t = [(\mathbf{h}_t^d)^\top \mathbf{h}_1^e, \dots, (\mathbf{h}_t^d)^\top \mathbf{h}_n^e] \in \mathbb{R}^n$$

- We take softmax to get the attention distribution  $\alpha_t$  for this step (this is a probability distribution and sums to 1)

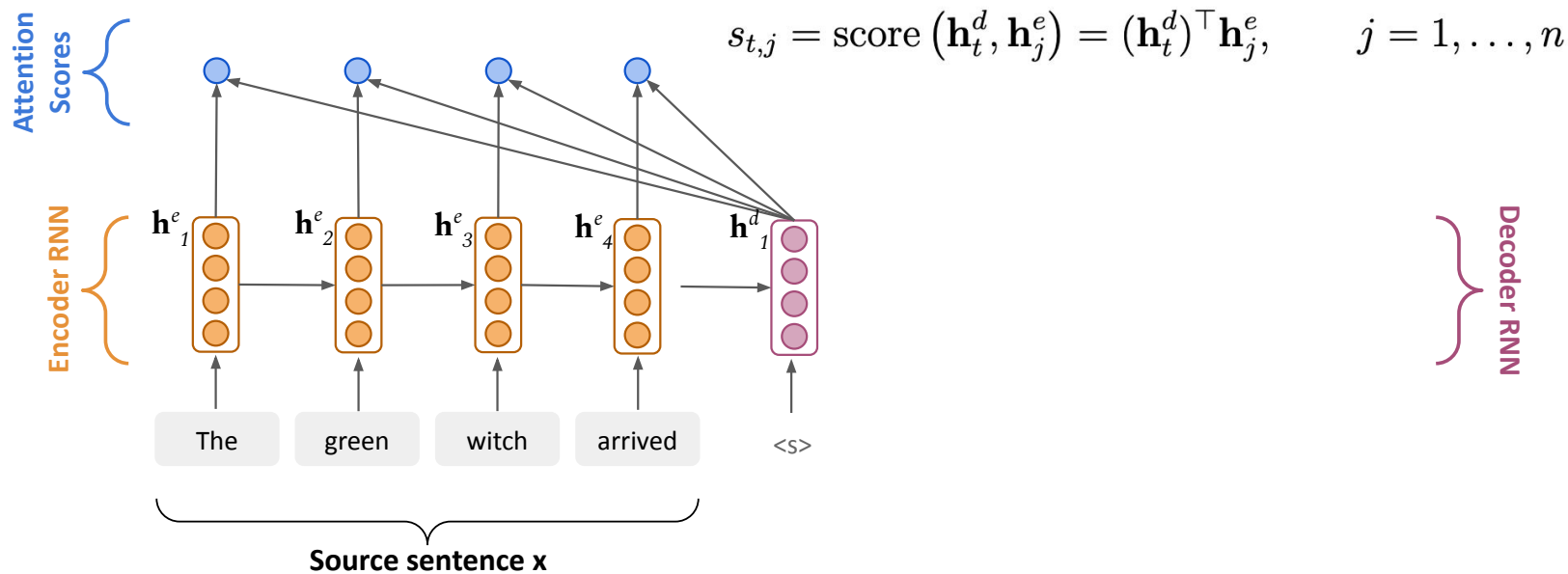
$$\alpha_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^n$$

- We use to take a weighted sum of the encoder hidden states to get the
- attention output

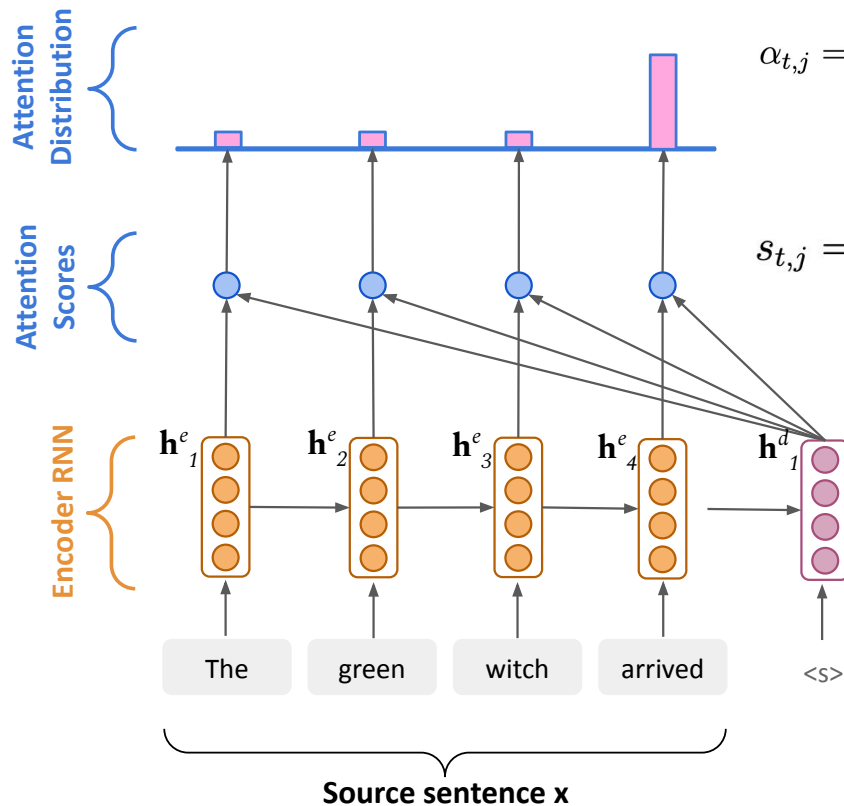
$$\mathbf{c}_t = \sum_{j=1}^n \alpha_{t,j} \mathbf{h}_j^e$$

- Finally we concatenate the attention output with the decoder hidden state and proceed as in the non-attention seq2seq model  $[\mathbf{h}_t^d; \mathbf{c}_t] \in \mathbb{R}^{2h}$

# Seq2seq with Attention



# Seq2seq with Attention



$$\alpha_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^n$$

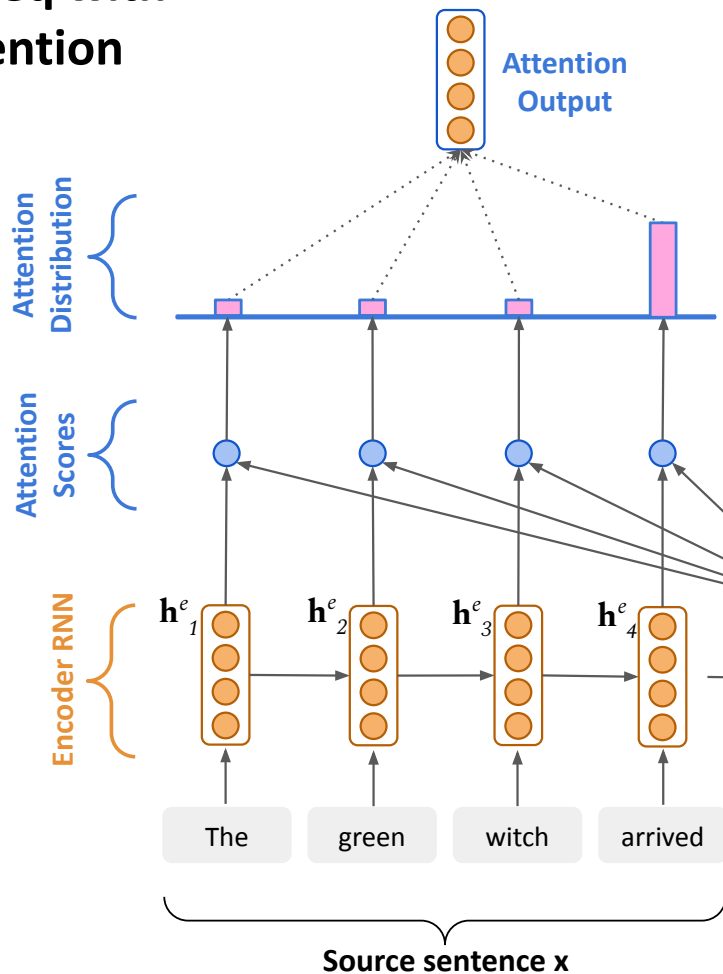
$$\alpha_{t,j} = \frac{\exp(s_{t,j})}{\sum_{k=1}^n \exp(s_{t,k})}$$

$$\alpha_{t,j} = \frac{\exp((\mathbf{h}_t^d)^\top \mathbf{h}_j^e)}{\sum_{k=1}^n \exp((\mathbf{h}_t^d)^\top \mathbf{h}_k^e)}$$

$$s_{t,j} = \text{score}(\mathbf{h}_t^d, \mathbf{h}_j^e) = (\mathbf{h}_t^d)^\top \mathbf{h}_j^e, \quad j = 1, \dots, n$$

Decoder RNN

# Seq2seq with Attention



$$\mathbf{c}_t = \sum_{j=1}^n \alpha_{t,j} \mathbf{h}_j^e$$

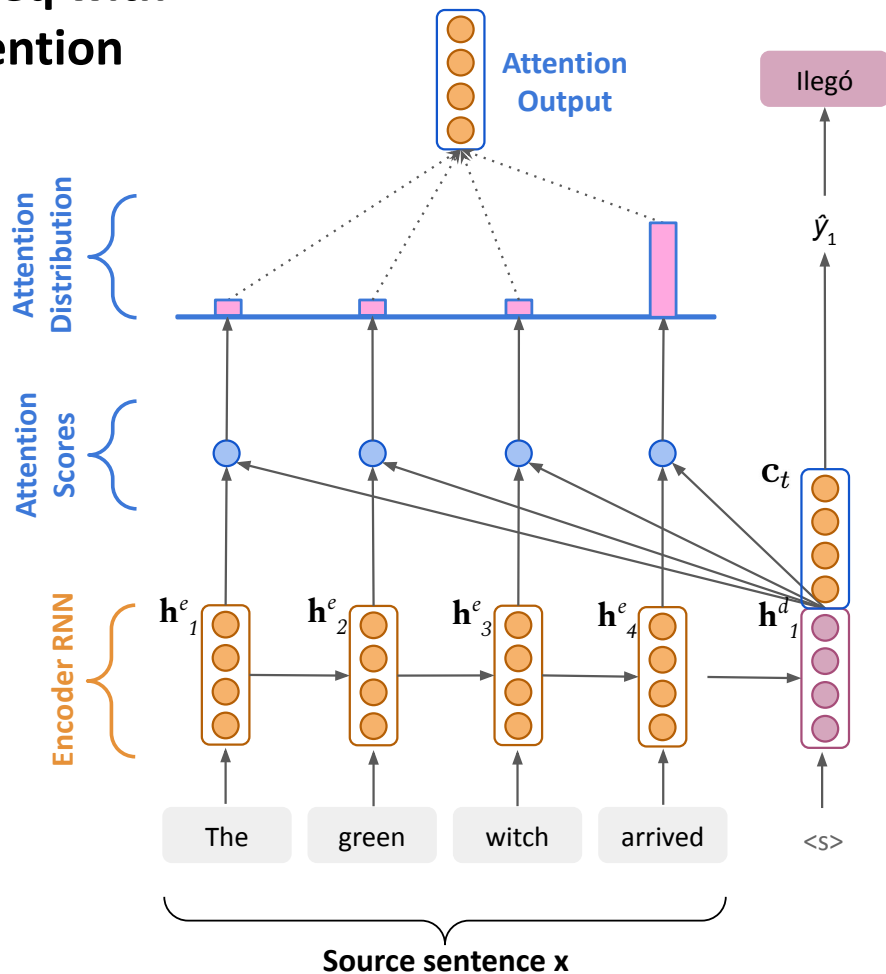
$$\boldsymbol{\alpha}_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^n$$

$$\alpha_{t,j} = \frac{\exp(s_{t,j})}{\sum_{k=1}^n \exp(s_{t,k})} \quad \alpha_{t,j} = \frac{\exp((\mathbf{h}_t^d)^\top \mathbf{h}_j^e)}{\sum_{k=1}^n \exp((\mathbf{h}_t^d)^\top \mathbf{h}_k^e)}$$

$$s_{t,j} = \text{score}(\mathbf{h}_t^d, \mathbf{h}_j^e) = (\mathbf{h}_t^d)^\top \mathbf{h}_j^e, \quad j = 1, \dots, n$$

Decoder RNN

# Seq2seq with Attention



$$\hat{y}_t = \text{softmax}(\mathbf{z}_t)$$

$$\mathbf{z}_t = U \begin{bmatrix} \mathbf{h}_t^d \\ \mathbf{c}_t \end{bmatrix} + \mathbf{b}_2$$

$$U \in \mathbb{R}^{|V| \times 2h}, \quad \mathbf{b}_2 \in \mathbb{R}^{|V|}, \quad \mathbf{z}_t \in \mathbb{R}^{|V|}$$

Decoder RNN

# Why Attention?

- Attention significantly **improves NMT performance**
  - It's very useful to allow decoder to focus on certain parts of the source
- Attention **solves the bottleneck problem**
  - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with **vanishing gradient problem**
  - Provides shortcut to faraway states
- Attention provides **some interpretability**
  - By inspecting attention distribution, we can see what the decoder was focusing on
- We get (soft) **alignment for free!**
  - This is cool because we never explicitly trained an alignment system
  - The network just learned alignment by itself

# Attention Variants

Multiple ways to  
compute this!

- Attention involves:

- Computing the *attention scores*:  $\mathbf{s}_t = [(\mathbf{h}_t^d)^\top \mathbf{h}_1^e, \dots, (\mathbf{h}_t^d)^\top \mathbf{h}_n^e] \in \mathbb{R}^n$
- Taking softmax to get *attention distribution*  $\alpha_t$ :  $\alpha_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^n$
- Using attention distribution to take weighted sum of values:  $\mathbf{c}_t = \sum_{j=1}^n \alpha_{t,j} \mathbf{h}_j^e$

thus obtaining the *attention output*  $\mathbf{c}_t$  (also called the context vector)

# Attention Variants

$$\mathbf{h}_j^e \in \mathbb{R}^{d_e}, \quad \mathbf{h}_t^d \in \mathbb{R}^{d_d}, \quad s_{t,j} \in \mathbb{R}$$

- **Basic dot product:**  $s_{t,j} = (\mathbf{h}_t^d)^\top \mathbf{h}_j^e$ 
  - Assumes  $(d_d = d_e)$
  - Version we saw earlier
- **Multiplicative:**  $s_{t,j} = (\mathbf{h}_t^d)^\top W_a \mathbf{h}_j^e$ 
  - Learns a linear transformation that determines how encoder and decoder states should be compared.
  - With learned weight matrix:  $W_a \in \mathbb{R}^{d_d \times d_e}$
- **Additive:**  $s_{t,j} = \mathbf{v}_a^\top \tanh(W_e \mathbf{h}_j^e + W_d \mathbf{h}_t^d)$ 
  - Uses a small neural network to learn a nonlinear compatibility function between encoder and decoder states.
  - With learned weights:  $W_e \in \mathbb{R}^{d_a \times d_e}, \quad W_d \in \mathbb{R}^{d_a \times d_d}, \quad \mathbf{v}_a \in \mathbb{R}^{d_a}$

## **TODOs:**

- **Homework 2 Due: 10/23**
- **Project Proposal Due: 10/02**

**Next Class: Language Generation**